

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – SHORT ANSWER TEST

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 3 – Short Answer Test
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	<i>Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.</i>		
Student Name:	K.Nilavan	Reg. No.:	192511028
Section / Batch:	2025 – 2029	Date:	02/09/2026

Code of Conduct

I, K.Nilavan (192511028) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K.Nilavan

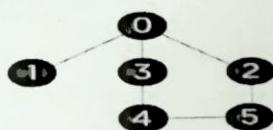
Instructions

- This test contains 10 short answer test questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20
Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
GRAND TOTAL:		100 Marks	

Annexure A - Short Answer Test

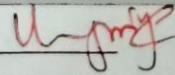
- Perform the following operations in the Singly Linked List for the following input.
Input: 22-> 77-> 55-> 88->
 - Insert a new-node 110 at the end of the list.
 - Insert a new-node 101 at the position 4 of the list.
 - Delete a node at the beginning of the single linked list
 - Delete a node at the position 3 of the single linked list.
- Rotations in each step of insertion. 1,2,3,4,5,6,7,8,9,10 (), 4) Push (90), 5) Push (36), 6) Push (11), 7) Push (88), 8) Pop (), 9) Pop (). Draw the diagram of the stack and illustrate the above operations and identify where the top is?
- Convert the following infix into a postfix expression. $\rightarrow B - (C/D + (E \% F * G) / H) * J$
- Evaluate the given postfix expression. $53+62/*35*+$
- Construct a B-Tree of Order 3 by inserting numbers 34,26,45,12,87,56,78,22,23,24,65,85,95.
- Construct a 2-3-4 Tree by inserting the following sequence of numbers 45, 36, 76, 23, 89, 115, 98, 39, 41, 56, 69, 48.
- Build a max heap for the following numbers - 20, 35, 23, 22, 4, 45, 21, 5, 42 and 19. At each stage of heap tree construction, ensure heap properties are satisfied.
- Perform the operations onto splay tree (draw after each operation): insert (3), insert (9), insert (12), insert (55), insert (1), insert (3), delete (3), delete (9), delete (60), insert (3), search (12).
- Describe the algorithm to sort the elements 170, 45, 75, 90, 802, 24, 2, 66 using Radix sort. Explain with each phase?
- Consider the following graph & construct BFS.



Graph with six nodes

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Conceptual Understanding	30	Demonstrates complete and accurate understanding of concepts	Good understanding with minor gaps	Basic understanding	Limited understanding
Accuracy of Answer	25	Answers are completely correct and relevant	Mostly correct with minor errors	Partially correct	Mostly incorrect
Problem-Solving & Reasoning	20	Provides logical reasoning and appropriate steps	Good reasoning with minor gaps	Basic reasoning	Lacks logical reasoning
Clarity & Relevance	15	Answers are precise, clear, concise, and directly address the question	Mostly clear and relevant	Somewhat unclear or incomplete	Irrelevant or unclear answers
Time Management & Completion	10	Completes all questions accurately within the time	Completes most questions on time	Requires additional time	Unable to complete the test

SFaculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Singly Linked List

* A singly linked list consists of nodes where each node contains data & pointer to next node.

given: $22 \rightarrow 77 \rightarrow 55 \rightarrow 88 \rightarrow \text{NULL}$

i) Insert 110 at end:

$22 \rightarrow 77 \rightarrow 55 \rightarrow 88 \rightarrow 110 \rightarrow \text{NULL}$.

ii) Insert 101 at position 4:

$22 \rightarrow 77 \rightarrow 55 \rightarrow 101 \rightarrow 88 \rightarrow 110 \rightarrow \text{NULL}$

iii) Delete at beginning:

$77 \rightarrow 55 \rightarrow 101 \rightarrow 88 \rightarrow 110 \rightarrow \text{NULL}$

iv) Delete at position 3:

$77 \rightarrow 55 \rightarrow 88 \rightarrow 110 \rightarrow \text{NULL}$

Thus, insertion and deletion are performed by modifying the links between nodes.

2) Stack Operations:

A stack is a linear data structure that follows LIFO (Last In, First Out).

Operations:

Push(90), Push(36), Push(11), Push(88), pop(), pop()

Operation	Stack
Push(90)	90
Push(36)	90, 36
Push(11)	90, 36, 11
Push(88)	90, 36, 11, 88
Pop()	90, 36, 11
Pop()	90, 36

Final Stack: 90, 36

TOP = 36.

* The elements removed are 88 and 11.

3) Infix to Prefix:

* An infix expression places operators between operands, while a postfix expression places operators after operands. A stack is used for conversion.

Given: $B \times (C/D + (E \% F + G) / H) + J$

Postfix expression:

$BCD / EF \% G + H / + J + -$

* Thus, the infix expression is successfully converted into postfix form.

4) Postfix Evaluation:

Postfix expressions are evaluated using a stack. Operands are pushed & operators perform operations on the top operands.

Ex: $53+62 / * 35 +$

Evaluation:

$$5+3 = 8$$

$$6/2 = 3$$

$$8 * 3 = 24$$

$$3 * 5 = 15$$

$$24 + 15 = 39$$

Final Answer: 39 //

5) B-Tree of Order 3

A B-tree is a balanced multiway search tree. In an order-3 B-tree, a node can have a maximum of 3 children & 2 keys. When a node becomes full, the middle key is promoted.

Ex:

34, 26, 145, 12, 87, 56, 78, 22, 23, 24, 65, 85, 95

* The insertion process involves splitting full nodes & promoting middle keys

* The resulting tree remains balanced, with keys in sorted order & all leaf nodes at the same level.

* B-Trees provide efficient searching & insertion by maintaining a balanced structure.

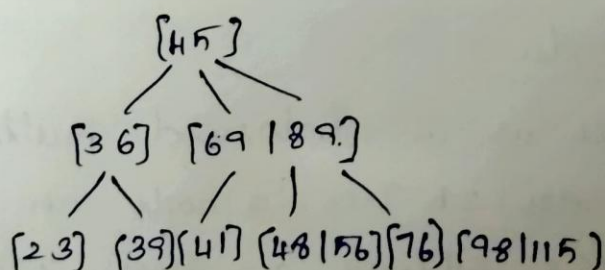
6) * A 2-3-4 Tree is a balanced search tree containing 2-nodes, 3-nodes & 4-nodes.

Given:

45, 36, 76, 23, 89, 115, 98, 39, 41, 56, 69, 48.

* During insertion, a full node is split & its middle key is promoted to maintain balance.

Final Structure:



Thus the tree remains balanced & maintains sorted ordering.

7) Max Heap:

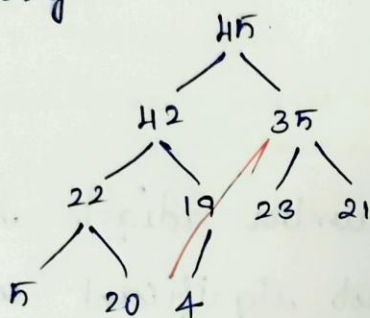
A Max Heap is a complete binary tree in

which every parent is greater than or equal to its children.

Given:

20, 35, 23, 22, 4, 45, 21, 5, 42, 19.

After inserting elements & maintaining the heap property:



Array representation: [45, 42, 35, 22, 19, 23, 21, 5, 20, 4].

+ since every parent is greater than its children, the Max Heap property is satisfied.

8) Splay Tree:

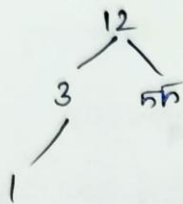
A Splay Tree is self-adjusting Binary Search tree. After an insertion, deletion & search, the accessed node is moved to root using rotations.

Operations given:

Insert(3), Insert(9), Insert(12), Insert(55), Insert(1),
Insert(3), Delete(3), Delete(9), Delete(60), Insert(3),
Search(12).

+ The tree is adjusted after every operation using Zig, Zig-Zig & Zig-Zag rotations.

After all operations, the final structure is:



* Delete (60) has no effect because 60 is not present.

9) Radix Sort

Radix Sort sorts number digit by digit, starting from the least significant digit.

Given:

170, 45, 75, 90, 802, 24, 2, 66.

Pass 1 - units digit (170, 90, 802, 2, 24, 45, 75, 66)

Pass 2 - Tens digit (802, 2, 24, 45, 66, 170, 75, 90)

Pass 3 - Hundreds digit

Final result : 2, 24, 45, 66, 75, 90, 170, 802.

10) Breadth First Search:

* BFS is graph traversal technique that visits vertices level by level. It uses queue & follows the FIFO principle.

BFS Traversal:

0 → 1 → 3 → 2 → 4 → 5

* It is useful for finding shortest paths in unweighted graphs, network traversal & connectivity analysis